

ПЛАН ВНЕДРЕНИЯ

Встроенный сертификат GigaChat

Отложенный этап после переноса React-frontend и подключения FastAPI

Результат этапа

Пользователь выбирает GigaChat, вводит ключ и работает без ручного скачивания или установки
российского корневого сертификата.

Версия 1.0 · 6 сентября 2026

1. Назначение и границы этапа

Документ описывает только внедрение встроенного доверенного сертификата для GigaChat. Работы начинаются после того, как frontend перенесён в проект, а backend-разработчик подключил рабочий FastAPI и экран настроек может обращаться к локальному API.

Не установка в Windows

Сертификат подключается только к TLS-контексту HTTP-клиента GigaChat. Системное хранилище сертификатов Windows не изменяется; права администратора не нужны.

Целевая схема

React frontend	Локальный FastAPI	GigaChatClient
Показывает статус и запускает проверку	Проверяет ресурс и возвращает безопасный результат	Использует отдельный SSLContext с PEM

Границы безопасности

- Не использовать verify=False ни в development, ни в production.
- Не выполнять импорт сертификата в Windows Certificate Store.
- Не отдавать frontend локальный путь к PEM, API-ключ, OAuth-токен или тело ответа провайдера.
- Ошибка сертификата отключает только GigaChat; остальные LLM-провайдеры продолжают работать.

2. Работы backend-разработчика

2.1. Ресурс и контроль целостности

- Поместить официальный PEM в `app/resources/certs/russian_trusted_root_ca.pem.crt`.
- Добавить манифест: официальный источник, дата получения, `subject/issuer`, срок действия и SHA-256 `936A43FEA6E8E525BCC0F81ACD9C3D21B4FC4B9B68ACEA7906D698005AFC6504`.
- Загружать ресурс через `importlib.resources`, без зависимости от текущей директории.
- Проверять наличие, SHA-256 и срок действия при запуске. За 30 дней до окончания возвращать статус `expiring`.

2.2. TLS-подключение

- Создать `ssl.create_default_context(cafile=...)` и передать контекст в отдельный `httpx.AsyncClient` `GigaChat`.
- Не применять этот CA к `OpenAI`, `Anthropic` или пользовательскому роутеру.
- Включить PEM и манифест в `package data/PyInstaller datas`; сборка должна завершаться ошибкой, если ресурс отсутствует.

2.3. Публичный API

Метод	Назначение	Ответ
GET <code>/api/config</code>	Публичная конфигурация и состояние <code>GigaChat</code>	<code>certificate.status</code> , <code>expires_at</code> ; без пути и секретов
POST <code>/api/config/validate</code>	Проверка сертификата, OAuth и модели без сохранения	<code>ok</code> , <code>code</code> , <code>message</code> , <code>certificate</code> ; без тела ответа

Безопасные коды

`ready`, `expiring`, `certificate_missing`, `certificate_integrity_failed`, `certificate_expired`, `authentication_failed`, `model_not_found`, `connection_failed`. Сообщения не содержат ключ, токен, путь или ответ `GigaChat`.

3. Работы frontend-разработчика

Интерфейс строится по данным backend. Frontend не читает PEM и не обращается к GigaChat напрямую.

3.1. Экран настроек GigaChat

- Показать поля «Ключ авторизации» и «Модель», а также отдельный блок состояния сертификата.
- Добавить кнопку «Проверить подключение», вызывающую POST /api/config/validate.
- После сохранения очищать поле ключа; не использовать localStorage, sessionStorage, URL, аналитику или клиентские логи.
- Не показывать кнопки «Скачать сертификат» и «Установить сертификат».

3.2. Тексты и состояния

Статус	Что показать пользователю	Действие
ready	Встроенный сертификат проверен и подключён к GigaChat	Разрешить проверку подключения
expiring	Сертификат работает, но приложению скоро потребуется обновление	Разрешить работу, показать предупреждение
missing / integrity_failed / expired	GigaChat недоступен: обновите или переустановите приложение	Заблокировать сохранение GigaChat
authentication_failed	Не удалось подтвердить ключ авторизации	Оставить форму для исправления

Важно для формулировок

Правильная формулировка: «сертификат встроен и подключён внутри приложения». Не писать, что сертификат установлен в Windows или внедрён во всю среду пользователя.

4. Проверка, передача и готовность

Backend-тесты

- Проверить корректный, отсутствующий, изменённый, истёкший и истекающий сертификат.
- Подтвердить, что GigaChat получает отдельный SSLContext, а verify=False отсутствует.
- Проверить разрешение ресурса из исходников и собранного приложения.
- Проверить, что API, исключения и логи не содержат секретов, локального пути и тела ответа провайдера.

Frontend-тесты

- Проверить отображение ready, expiring и недоступных состояний.
- Проверить очистку ключа после сохранения и отсутствие прямых внешних HTTP-запросов.
- Проверить независимость остальных провайдеров от состояния сертификата GigaChat.

Критерий сквозной приёмки

Чистая Windows-среда

На компьютере без русского корневого сертификата пользователь выбирает GigaChat, видит подтверждение встроенного сертификата, вводит ключ, успешно проверяет OAuth и выполняет синтетическое задание. Ручное скачивание, права администратора и изменение Windows Certificate Store не требуются.

Порядок запуска работ

1	Frontend перенесён в репозиторий; FastAPI и GET/PUT /api/config работают.
2	Backend добавляет ресурс, проверку целостности, TLS-контекст и validate API.
3	Frontend подключает статусы и кнопку проверки без работы с PEM.
4	Команда выполняет unit, packaging и сквозную проверку на чистой Windows.

Перед каждым релизом владелец сертификата проверяет срок действия и планирует замену не позднее чем за 30 дней до окончания.